Міністерство освіти та науки України
Харківський національний університет радіоелектроніки
Кафедра прикладної математики

Звіт
з лабораторної роботи №1
з дисципліни «Задачі та методи Data Mining»

Виконав:
ст. гр. ПМм-26-1
Серопян А. С.

Перевірив:
ст. викл. каф. ПМ
Янбеков Р.Я.

Харків 2026

1 НОРМИ L1, L2 ТА L3 І РОБОТА З БІБЛІОТЕКАМИ PYTHON

1.1 Мета роботи

Навчитися обчислювати норми L1, L2 та L3 за допомогою NumPy, SciPy, scikit-learn і PyTorch; порівняти результати бібліотек на спільних даних; дослідити вплив масштабування ознак на оцінювання спостережень.

1.2 Норми Lp і відстань Мінковського

Для вектора $x = (x_1; \ldots; x_n)^T$ і $p \geq 1$ норма порядку p визначається формулою

$$\|x\|_p = \left(\sum_{i=1}^{n} |x_i|^p \right)^{1/p}, \tag{1.1}$$

де x_i – координати вектора;

n – розмірність вектора;

p – порядок норми.

За $p = 1$ формула (1.1) дає суму модулів координат, за $p = 2$ – евклідову довжину вектора, за $p = 3$ – норму, у якій відносно більший внесок мають координати з більшим модулем. Зі зростанням p норма наближається до найбільшого модуля координати.

Відстань Мінковського між векторами x і y має вигляд

$$d_p(x, y) = \left(\sum_{i=1}^{n} |x_i - y_i|^p \right)^{1/p}, \tag{1.2}$$

де y_i – координати другого вектора.

Якщо y – нульовий вектор, відстань (1.2) збігається з нормою (1.1), тому функцію `pairwise_distances` зі scikit-learn можна використати для обчислення норм рядків матриці.

1.3 Масштабування та центрування

Масштабування `MinMaxScaler` переводить кожну ознаку в діапазон $[a; b]$:

$$x'_{ij} = \frac{x_{ij} - \min\limits_k x_{kj}}{\max\limits_k x_{kj} - \min\limits_k x_{kj}} \cdot (b - a) + a\,, \tag{1.3}$$

де x_{ij} – значення ознаки j у спостереженні i;

x'_{ij} – масштабоване значення;

a, b – межі цільового діапазону.

Центрування віднімає від кожної ознаки її середнє значення:

$$\hat{x}_{ij} = x'_{ij} - \bar{x}'_j, \quad \bar{x}'_j = \frac{1}{N}\sum_{k=1}^{N} x'_{kj}\,, \tag{1.4}$$

де $\hat{x}_{ij}$ – центроване значення ознаки;

$\bar{x}'_j$ – середнє значення ознаки j;

N – кількість спостережень.

Після центрування норма рядка показує віддаленість спостереження від середнього значення вибірки.

1.4 Програмне забезпечення

Роботу виконано у віртуальному середовищі `.venv`, створеному на основі стабільного Python 3.14.7. Використано такі бібліотеки:

– NumPy 2.5.4 – робота з масивами та функція `numpy.linalg.norm`;

– SciPy 1.18.1 – функція `scipy.linalg.norm`;

– scikit-learn 1.9.1 – `MinMaxScaler`, `pairwise_distances` і набір даних Iris;

– PyTorch 2.14.1 (CPU) – функція `torch.linalg.vector_norm`;

– pytest 9.2.0 – автоматична перевірка реалізації.

1.5 Обчислення норм вручну

Для вектора $x = (3; -4; 12)^T$ підставимо координати у формулу (1.1).

Норма L1:

$$\|x\|_1 = |3| + |-4| + |12| = 3 + 4 + 12 = 19.$$

Норма L2:

$$\|x\|_2 = \sqrt{3^2 + (-4)^2 + 12^2} = \sqrt{9 + 16 + 144} = \sqrt{169} = 13.$$

Норма L3:

$$\|x\|_3 = \sqrt[3]{|3|^3 + |-4|^3 + |12|^3} = \sqrt[3]{27 + 64 + 1728} = \sqrt[3]{1819} \approx 12,2071.$$

Аналогічно для другого тестового вектора $v = (1; -2; 2)^T$ отримаємо $\|v\|_1 = 5$, $\|v\|_2 = \sqrt{9} = 3$, $\|v\|_3 = \sqrt[3]{17} \approx 2,5713$.

Ручні значення збігаються з контрольними (L1 = 19, L2 = 13, L3 ≈ 12,2071) та з виводом програми (див. табл. 1.1). Порядок норм $\|x\|_1 > \|x\|_2 > \|x\|_3$ ілюструє, що зі зростанням p норма наближається до найбільшої за модулем координати, яка дорівнює 12.

1.6 Власна реалізація норми за допомогою NumPy

Функція `manual_lp_norm_numpy` реалізує формулу (1.1) операціями `np.abs`, піднесення до степеня та `np.sum`. Перед обчисленням функція перевіряє, що порядок норми належить множині $\{1,2,3\}$, а вхідний масив одновимірний; інакше генерується виняток `ValueError`:

```
def manual_lp_norm_numpy(vector: np.ndarray, order: int) -> float:
    """Обчислює норму вектора за формулою за допомогою NumPy."""
    validate_order(order)
    if vector.ndim != 1:
        raise ValueError("Очікувався одновимірний вектор.")

    # Сума модулів координат у степені p, з якої добувається корінь степеня p.
    powered_sum = np.sum(np.abs(vector) ** order)
    return float(powered_sum ** (1.0 / order))
```

Норми рядків матриці обчислюються функцією `row_norms_numpy` викликом `np.linalg.norm(data, ord=order, axis=1)`. Для векторів x і v власна функція повністю збіглася з `numpy.linalg.norm` (табл. 1.1).

1.7 Перевірка за допомогою SciPy

Функція `vector_norm_scipy` обчислює норму одного вектора викликом `scipy.linalg.norm(vector, ord=order)`, а функція `row_norms_scipy` — норму кожного рядка викликом `scipy.linalg.norm(data, ord=order, axis=1)`. Для тестових векторів SciPy дав ті самі значення, що й NumPy. Порівняння

виконувалося функцією `numpy.testing.assert_allclose` з допуском 10^{-6}, і всі перевірки пройдено.

Таблиця 1.1 – Норми тестових векторів у різних реалізаціях

Реалізація	L1 для x	L2 для x	L3 для x	L1 для v	L2 для v	L3 для v
власна NumPy	19,000000	13,000000	12,207055	5,000000	3,000000	2,571282
`numpy.linalg.norm`	19,000000	13,000000	12,207055	5,000000	3,000000	2,571282
`scipy.linalg.norm`	19,000000	13,000000	12,207055	5,000000	3,000000	2,571282
`pairwise_distances`	19,000000	13,000000	12,207055	5,000000	3,000000	2,571282
`torch.linalg.vector_norm`	19,000000	13,000000	12,207055	5,000000	3,000000	2,571282
власна PyTorch	19,000000	13,000000	12,207055	5,000000	3,000000	2,571282

1.8 Масштабування та відстані у scikit-learn

Для експериментів використано дві ознаки пелюстки набору Iris: довжину (від 1,0 до 6,9 см) та ширину (від 0,1 до 2,5 см). Функція `scale_features` створює `MinMaxScaler` із заданим `feature_range` і викликає `fit_transform`, тобто реалізує формулу (1.3). Функція `row_norms_sklearn` створює нульовий вектор потрібної ширини та обчислює відстань до нього за допомогою `pairwise_distances(data, origin, metric="minkowski", p=order)`.

Масштабування виконано для двох діапазонів: $[0;1]$ та $[-1;1]$. Рядки з найбільшими оцінками для обох діапазонів наведено в табл. 1.2, значення ознак ключових спостережень – у табл. 1.3.

Таблиця 1.2 – П’ять найбільших норм L2 для різних діапазонів масштабування

Місце	Індекс, [0, 1]	Вид	Норма L2	Індекс, [-1, 1]	Вид	Норма L2
1	118	virginica	1,3566	13	setosa	1,3905
2	109	virginica	1,3218	22	setosa	1,3566
3	100	virginica	1,3108	37	setosa	1,3218
4	117	virginica	1,3034	12	setosa	1,3218
5	144	virginica	1,2785	14	setosa	1,3074

Для норм L1 і L3 порядок п’яти перших спостережень в обох діапазонах такий самий, як для L2. Спостереження 37 та 12 мають однакові значення ознак, тому їхні норми збігаються.

Таблиця 1.3 – Значення ознак ключових спостережень Iris

Індекс	Вид	Довжина, см	Ширина, см	Діапазон [0, 1]	Діапазон [-1, 1]	Центровані
13	setosa	1,1	0,1	(0,0169; 0,0000)	(-0,9661; -1,0000)	(-0,4505; -0,4581)
22	setosa	1,0	0,2	(0,0000; 0,0417)	(-1,0000; -0,9167)	(-0,4675; -0,4164)
100	virginica	6,0	2,5	(0,8475; 1,0000)	(0,6949; 1,0000)	(0,3800; 0,5419)
109	virginica	6,1	2,5	(0,8644; 1,0000)	(0,7288; 1,0000)	(0,3969; 0,5419)
117	virginica	6,7	2,2	(0,9661; 0,8750)	(0,9322; 0,7500)	(0,4986; 0,4169)
118	virginica	6,9	2,3	(1,0000; 0,9167)	(1,0000; 0,8333)	(0,5325; 0,4586)
144	virginica	5,7	2,5	(0,7966; 1,0000)	(0,5932; 1,0000)	(0,3292; 0,5419)

1.9 Обчислення за допомогою PyTorch

Тестовий вектор і масштабовані дані перетворено на тензори типу `torch.float64` функцією `torch.as_tensor`. Норми вектора обчислено методом `torch.linalg.vector_norm(tensor, ord=p)`, а норми рядків – функцією `row_norms_torch` із параметрами `ord=order` та `dim=1`. Власна функція `manual_lp_norm_torch` реалізує формулу (1.1) засобами PyTorch:

```
def manual_lp_norm_torch(vector: torch.Tensor, order: int) -> torch.Tensor:
    """Обчислює норму тензора за формулою за допомогою PyTorch."""
    validate_order(order)
    if vector.ndim != 1:
        raise ValueError("Очікувався одновимірний вектор.")

    powered_sum = torch.sum(torch.abs(vector) ** order)
    return powered_sum ** (1.0 / order)
```

Результати власної функції, `torch.linalg.vector_norm` і NumPy для векторів x і v збіглися (табл. 1.1).

1.10 Порівняння бібліотек і вплив центрування

Функція `compare_libraries` для кожного $p \in \{1,2,3\}$ обчислює норми рядків у NumPy, SciPy, scikit-learn і PyTorch, перевіряє збіг із NumPy функцією `numpy.testing.assert_allclose` з допуском 10^{-6} та виводить індекс рядка з найбільшою оцінкою. Результати для масштабованих і центрованих даних Iris наведено в табл. 1.4.

Таблиця 1.4 – Рядок із найбільшою нормою та його оцінка в чотирьох бібліотеках

Дані	Норма	Рядок	NumPy	SciPy	scikit-learn	PyTorch
[0, 1]	L1	118	1,9167	1,9167	1,9167	1,9167
[0, 1]	L2	118	1,3566	1,3566	1,3566	1,3566
[0, 1]	L3	118	1,2097	1,2097	1,2097	1,2097
[-1, 1]	L1	13	1,9661	1,9661	1,9661	1,9661
[-1, 1]	L2	13	1,3905	1,3905	1,3905	1,3905
[-1, 1]	L3	13	1,2389	1,2389	1,2389	1,2389
центровані	L1	118	0,9912	0,9912	0,9912	0,9912
центровані	L2	118	0,7028	0,7028	0,7028	0,7028
центровані	L3	118	0,6278	0,6278	0,6278	0,6278

Для всіх 150 спостережень і всіх трьох норм максимальна абсолютна різниця між результатами SciPy, scikit-learn, PyTorch і NumPy дорівнює нулю, тобто не перевищує допуску 10^{-6}. Для обчислень у PyTorch з типом `float32` різниця з NumPy становила близько 10^{-7}.

Центрування за формулою (1.4) виконано відніманням середнього значення кожної масштабованої ознаки (0,4675 для довжини та 0,4581 для

ширини, що відповідає 3,758 см та 1,199 см у вихідних одиницях). Рядок із найбільшою оцінкою після центрування залишився тим самим, що й для діапазону $[0;1]$ (спостереження 118), але зміст оцінки змінився: тепер вона показує відхилення від середньої квітки, а не від найменшої. Значення норм зменшилися приблизно вдвічі, оскільки середнє лежить усередині діапазону даних.

1.11 Аналіз набору Iris

Для центрованих ознак пелюстки обчислено норми L1, L2 і L3 кожного спостереження чотирма бібліотеками; результати збіглися з NumPy з точністю 10^{-6}. Функція `top_five_observations` упорядковує оцінки функцією `numpy.argsort`, вибирає п’ять найбільших у порядку спадання та повертає для кожної індекс спостереження, назву виду й значення норми. Результати наведено в табл. 1.5.

Таблиця 1.5 – П’ять найбільших відхилень від середнього в наборі Iris

Місце	Індекс L1 (вид)	Норма L1	Індекс L2 (вид)	Норма L2	Індекс L3 (вид)	Норма L3
1	118 (virginica)	0,9912	118 (virginica)	0,7028	118 (virginica)	0,6278
2	109 (virginica)	0,9389	109 (virginica)	0,6718	109 (virginica)	0,6052
3	100 (virginica)	0,9219	100 (virginica)	0,6619	100 (virginica)	0,5982
4	117 (virginica)	0,9156	117 (virginica)	0,6500	117 (virginica)	0,5813
5	13 (setosa)	0,9086	13 (setosa)	0,6425	144 (virginica)	0,5797

Перші чотири позиції однакові для всіх норм – це найбільші квітки виду virginica. Відмінність виникає лише на п’ятому місці. Спостереження 13 має

майже однакові за модулем відхилення за обома ознаками $(-0,4505; -0,4581)$, а спостереження 144 – одне велике відхилення за шириною (0,5419) і менше за довжиною (0,3292). Норма L1 підсумовує відхилення і тому надає перевагу збалансованому відхиленню за обома ознаками: 0,9086 для спостереження 13 проти 0,8711 для спостереження 144. Норма L3 сильніше реагує на найбільшу координату, тому спостереження 144 з оцінкою 0,5797 випереджає спостереження 13 з оцінкою 0,5724.

Отримане ранжування показує лише віддаленість від середнього значення двох ознак. Усі знайдені спостереження є коректними квітками, які лежать на краях своїх видів; набір Iris складається з трьох груп, і середнє значення розташоване між ними, тому велика норма тут відображає належність до крайньої групи, а не помилку в даних.

1.12 Перевірка тестами

Функції реалізовано у файлі `student_implementation.py`, створеному копіюванням `student_template.py`. Для кожного методу у файлі `tests/test_student_implementation.py` написано власні тести. Очікувані значення не дублюються в коді тестів, а надходять із фікстур файлу `tests/conftest.py`: `reference_lp_norm`, `reference_row_norms` і `reference_min_max_scale` обчислюють еталон незалежно від бібліотек, що перевіряються, а `vector_case`, `matrix_data`, `norm_order`, `feature_range`, `invalid_norm_order`, `iris_dataset` і `centered_iris_features` задають вхідні дані.

Таблиця 1.7 – Результат запуску pytest -v

Файл тестів	Призначення	Пройдено	Пропущено
`tests/test_implementation.py`	перевірка еталонного рішення `solution.py`	122	0
`tests/test_template.py`	базова перевірка `student_implementation.py`	19	0
`tests/test_student_implementation.py`	власні тести	60	0
Разом		201	0

Висновки

У лабораторній роботі реалізовано обчислення норм L1, L2 та L3 за власною формулою засобами NumPy і PyTorch, а також за допомогою вбудованих функцій NumPy, SciPy, scikit-learn і PyTorch. Ручний розрахунок для вектора $(3; -4; 12)$ дав значення 19, 13 та 12,2071, які збіглися з результатами всіх шести реалізацій.

На масштабованих і центрованих ознаках пелюстки набору Iris усі чотири бібліотеки дали однакові норми для всіх 150 спостережень: різниця з NumPy для типу `float64` дорівнює нулю, що значно менше допуску 10^{-6}. Реалізацію підтверджено повним набором із 201 автоматичного тесту, серед яких 60 запусків власних тестів; усі тести пройдено без пропусків.

Отже, норми рядків є зручною та відтворюваною мірою величини спостереження, але її результат залежить від масштабу, центрування та порядку норми, тому без урахування структури даних вона не може слугувати надійним детектором викидів.

Додаток А. Код програми student_implementation.py

```
"""Шаблон для самостійного виконання лабораторної роботи № 1."""

from __future__ import annotations

import numpy as np
import torch
from scipy.linalg import norm as scipy_norm
from sklearn.datasets import load_iris
from sklearn.metrics import pairwise_distances
from sklearn.preprocessing import MinMaxScaler

NORM_ORDERS = (1, 2, 3)
TOLERANCE = 1e-6


def validate_order(order: int) -> None:
    """Перевіряє порядок норми."""
    if order not in NORM_ORDERS:
        raise ValueError("Порядок норми має дорівнювати 1, 2 або 3.")


def manual_lp_norm_numpy(vector: np.ndarray, order: int) -> float:
    """Обчислює норму вектора за формулою за допомогою NumPy."""
    validate_order(order)
    if vector.ndim != 1:
        raise ValueError("Очікувався одновимірний вектор.")

    # Сума модулів координат у степені p, з якої добувається корінь степеня p.
    powered_sum = np.sum(np.abs(vector) ** order)
    return float(powered_sum ** (1.0 / order))


def row_norms_numpy(data: np.ndarray, order: int) -> np.ndarray:
    """Обчислює норму кожного рядка за допомогою NumPy."""
    validate_order(order)
    if data.ndim != 2:
        raise ValueError("Очікувалася двовимірна матриця.")

    return np.linalg.norm(data, ord=order, axis=1)


def row_norms_scipy(data: np.ndarray, order: int) -> np.ndarray:
    """Обчислює норму кожного рядка за допомогою SciPy."""
    validate_order(order)
    if data.ndim != 2:
        raise ValueError("Очікувалася двовимірна матриця.")

    return scipy_norm(data, ord=order, axis=1)


def vector_norm_scipy(vector: np.ndarray, order: int) -> float:
    """Обчислює норму вектора за допомогою SciPy."""
    validate_order(order)
    if vector.ndim != 1:
        raise ValueError("Очікувався одновимірний вектор.")

    return float(scipy_norm(vector, ord=order))


def scale_features(
```

```
    data: np.ndarray,
    feature_range: tuple[float, float],
) -> np.ndarray:
    """Масштабує ознаки за допомогою scikit-learn."""
    scaler = MinMaxScaler(feature_range=feature_range)
    return scaler.fit_transform(data)


def row_norms_sklearn(data: np.ndarray, order: int) -> np.ndarray:
    """Обчислює відстань рядків до нуля через scikit-learn."""
    validate_order(order)
    if data.ndim != 2:
        raise ValueError("Очікувалася двовимірна матриця.")

    # Нульовий вектор має ту саму кількість ознак, що й дані.
    origin = np.zeros((1, data.shape[1]), dtype=data.dtype)
    distances = pairwise_distances(data, origin, metric="minkowski", p=order)
    return distances.ravel()


def manual_lp_norm_torch(vector: torch.Tensor, order: int) -> torch.Tensor:
    """Обчислює норму тензора за формулою за допомогою PyTorch."""
    validate_order(order)
    if vector.ndim != 1:
        raise ValueError("Очікувався одновимірний вектор.")

    powered_sum = torch.sum(torch.abs(vector) ** order)
    return powered_sum ** (1.0 / order)


def row_norms_torch(data: torch.Tensor, order: int) -> torch.Tensor:
    """Обчислює норму кожного рядка за допомогою PyTorch."""
    validate_order(order)
    if data.ndim != 2:
        raise ValueError("Очікувалася двовимірна матриця.")

    return torch.linalg.vector_norm(data, ord=order, dim=1)


def compare_libraries(data: np.ndarray, title: str) -> dict[int, np.ndarray]:
    """Обчислює й порівнює норми рядків у чотирьох бібліотеках."""
    print(f"\n{title}")
    results_by_order: dict[int, np.ndarray] = {}

    for order in NORM_ORDERS:
        tensor = torch.as_tensor(data.copy(), dtype=torch.float64)
        scores = {
            "NumPy": row_norms_numpy(data, order),
            "SciPy": row_norms_scipy(data, order),
            "scikit-learn": row_norms_sklearn(data, order),
            "PyTorch": row_norms_torch(tensor, order).cpu().numpy(),
        }

        # Відмінність кожної бібліотеки від NumPy не має перевищувати 1e-6.
        numpy_scores = scores["NumPy"]
        print(f"  Норма L{order}:")
        for library, library_scores in scores.items():
            np.testing.assert_allclose(
                library_scores,
                numpy_scores,
                rtol=0.0,
                atol=TOLERANCE,
            )
            top_index = int(np.argmax(library_scores))
```

```
            max_difference = float(np.max(np.abs(library_scores -
numpy_scores)))
            print(
                f"    {library}: рядок={top_index}, "
                f"оцінка={library_scores[top_index]:.4f}, "
                f"різниця з NumPy={max_difference:.2e}"
            )
        results_by_order[order] = numpy_scores

    return results_by_order


def top_five_observations(
    scores: np.ndarray,
    labels: np.ndarray,
    target_names: np.ndarray,
) -> list[tuple[int, str, float]]:
    """Повертає індекс, назву виду й оцінку для п'яти найбільших норм."""
    if scores.ndim != 1 or labels.ndim != 1:
        raise ValueError("Оцінки й мітки мають бути одновимірними масивами.")
    if len(scores) != len(labels):
        raise ValueError("Кількість оцінок має збігатися з кількістю міток.")

    # Індекси п'яти найбільших оцінок у порядку спадання.
    largest_indices = np.argsort(scores)[-5:][::-1]
    return [
        (int(index), str(target_names[labels[index]]), float(scores[index]))
        for index in largest_indices
    ]


def print_vector_norms(vector: np.ndarray) -> None:
    """Порівнює норми вектора в NumPy, SciPy, scikit-learn і PyTorch."""
    tensor = torch.as_tensor(vector.copy(), dtype=torch.float64)
    print(f"\nВектор {vector.tolist()}:")
    for order in NORM_ORDERS:
        values = {
            "власна NumPy": manual_lp_norm_numpy(vector, order),
            "NumPy": float(np.linalg.norm(vector, ord=order)),
            "SciPy": vector_norm_scipy(vector, order),
            "scikit-learn": float(
                row_norms_sklearn(vector.reshape(1, -1), order)[0]
            ),
            "PyTorch": torch.linalg.vector_norm(tensor, ord=order).item(),
            "власна PyTorch": manual_lp_norm_torch(tensor, order).item(),
        }
        np.testing.assert_allclose(
            list(values.values()),
            values["NumPy"],
            rtol=0.0,
            atol=TOLERANCE,
        )
        formatted = ", ".join(f"{name}={value:.6f}" for name, value in
values.items())
        print(f"  L{order}: {formatted}")


def main() -> None:
    """Виконує завдання лабораторної роботи в заданій послідовності."""
    # Завдання 1: ручний розрахунок для (3, -4, 12) дає L1=19, L2=13,
L3≈12.2071.
    sample = np.array([3.0, -4.0, 12.0])
    print("Контрольний вектор:", sample)
```

```
    # Завдання 2: власна функція NumPy порівнюється з np.linalg.norm.
    for order in NORM_ORDERS:
        print(f"L{order} NumPy:", np.linalg.norm(sample, ord=order))
        print(f"L{order} власна реалізація:", manual_lp_norm_numpy(sample,
order))

    # Завдання 3: норма вектора SciPy.
    for order in NORM_ORDERS:
        print(f"L{order} SciPy:", vector_norm_scipy(sample, order))

    # Завдання 5: тензор із sample, вбудована та власна функції PyTorch.
    sample_tensor = torch.as_tensor(sample.copy(), dtype=torch.float64)
    for order in NORM_ORDERS:
        print(
            f"L{order} PyTorch:",
            torch.linalg.vector_norm(sample_tensor, ord=order).item(),
        )
        print(
            f"L{order} власна реалізація PyTorch:",
            manual_lp_norm_torch(sample_tensor, order).item(),
        )

    # Завдання 2–5: порівняння тестових векторів у всіх бібліотеках.
    for vector in (sample, np.array([1.0, -2.0, 2.0])):
        print_vector_norms(vector)

    iris = load_iris()
    features = iris.data[:, 2:4]

    # Завдання 4: порівняння результатів для обох діапазонів масштабування.
    scaled = scale_features(features, feature_range=(0.0, 1.0))
    compare_libraries(scaled, "Масштабовані ознаки Iris, діапазон [0, 1]")
    scaled_signed = scale_features(features, feature_range=(-1.0, 1.0))
    compare_libraries(scaled_signed, "Масштабовані ознаки Iris, діапазон [-1,
1]")

    # Завдання 6: центрування масштабованих ознак відносно середнього.
    centered = scaled - scaled.mean(axis=0)
    centered_scores = compare_libraries(centered, "Центровані ознаки Iris")

    # Завдання 7: п'ять найбільших відхилень від середнього для кожної норми.
    for order, scores in centered_scores.items():
        print(f"\nП'ять найбільших відхилень для L{order}:")
        observations = top_five_observations(
            scores,
            iris.target,
            iris.target_names,
        )
        for index, species, score in observations:
            print(
                f"індекс={index}, вид={species}, норма={score:.4f}"
            )


if __name__ == "__main__":
    main()
```

Додаток Б. Код тестів test_student_implementation.py

```
"""Власні тести для реалізації лабораторної роботи № 1."""

from __future__ import annotations

from typing import Any, Callable, Iterable

import numpy as np
import pytest
import torch

RELATIVE_TOLERANCE = 1e-6
ABSOLUTE_TOLERANCE = 1e-8
LIBRARIES = ("NumPy", "SciPy", "scikit-learn", "PyTorch")


def load_student_implementation() -> Any:
    """Завантажує реалізацію студента або пропускає тест до її створення."""
    return pytest.importorskip(
        "student_implementation",
        reason="Створіть student_implementation.py на основі
student_template.py.",
    )


def assert_close(actual: Any, expected: Any) -> None:
    """Порівнює результати зі спільними для всіх тестів допусками."""
    np.testing.assert_allclose(
        actual,
        expected,
        rtol=RELATIVE_TOLERANCE,
        atol=ABSOLUTE_TOLERANCE,
    )


def to_tensor(values: np.ndarray) -> torch.Tensor:
    """Створює незалежний від масиву NumPy тензор подвійної точності."""
    return torch.as_tensor(values.copy(), dtype=torch.float64)


def test_manual_lp_norm_numpy(
    vector_case: np.ndarray,
    norm_order: int,
    reference_lp_norm: Callable[[Iterable[float], int], float],
) -> None:
    """Перевіряє ручну Lp-норму NumPy за еталонною фікстурою."""
    student = load_student_implementation()
    original = vector_case.copy()

    result = student.manual_lp_norm_numpy(vector_case, norm_order)

    assert isinstance(result, float)
    assert result == pytest.approx(reference_lp_norm(vector_case, norm_order))
    np.testing.assert_array_equal(vector_case, original)


def test_row_norms_numpy(
    matrix_data: np.ndarray,
    norm_order: int,
    reference_row_norms: Callable[[np.ndarray, int], np.ndarray],
) -> None:
```

```
    """Перевіряє норми рядків NumPy для кожного підтримуваного порядку."""
    student = load_student_implementation()

    result = student.row_norms_numpy(matrix_data, norm_order)

    assert result.shape == (matrix_data.shape[0],)
    assert_close(result, reference_row_norms(matrix_data, norm_order))


def test_row_norms_scipy(
    matrix_data: np.ndarray,
    norm_order: int,
    reference_row_norms: Callable[[np.ndarray, int], np.ndarray],
) -> None:
    """Перевіряє, що SciPy обчислює норми саме рядків, а не стовпців."""
    student = load_student_implementation()

    result = student.row_norms_scipy(matrix_data, norm_order)

    # Матриця 4x3: вісь 0 дала б три значення замість чотирьох.
    assert result.shape == (matrix_data.shape[0],)
    assert_close(result, reference_row_norms(matrix_data, norm_order))


def test_vector_norm_scipy(
    vector_case: np.ndarray,
    norm_order: int,
    reference_lp_norm: Callable[[Iterable[float], int], float],
) -> None:
    """Порівнює норму одного вектора SciPy з еталонною фікстурою."""
    student = load_student_implementation()

    result = student.vector_norm_scipy(vector_case, norm_order)

    assert isinstance(result, float)
    assert result == pytest.approx(reference_lp_norm(vector_case, norm_order))


def test_scale_features(
    matrix_data: np.ndarray,
    feature_range: tuple[float, float],
    reference_min_max_scale: Callable[
        [np.ndarray, tuple[float, float]], np.ndarray
    ],
) -> None:
    """Перевіряє MinMax-масштабування, зокрема сталу ознаку."""
    student = load_student_implementation()
    lower_bound, upper_bound = feature_range
    original = matrix_data.copy()

    result = student.scale_features(matrix_data, feature_range)

    assert_close(result, reference_min_max_scale(matrix_data, feature_range))
    # Змінні ознаки займають увесь діапазон, а стала переходить у нижню межу.
    assert_close(result[:, :2].min(axis=0), [lower_bound, lower_bound])
    assert_close(result[:, :2].max(axis=0), [upper_bound, upper_bound])
    assert_close(result[:, 2], np.full(matrix_data.shape[0], lower_bound))
    np.testing.assert_array_equal(matrix_data, original)


def test_row_norms_sklearn(
    matrix_data: np.ndarray,
    norm_order: int,
    reference_row_norms: Callable[[np.ndarray, int], np.ndarray],
```

```
) -> None:
    """Перевіряє відстані Мінковського від рядків до нуля."""
    student = load_student_implementation()

    result = student.row_norms_sklearn(matrix_data, norm_order)

    assert result.shape == (matrix_data.shape[0],)
    assert_close(result, reference_row_norms(matrix_data, norm_order))


def test_manual_lp_norm_torch(
    vector_case: np.ndarray,
    norm_order: int,
    reference_lp_norm: Callable[[Iterable[float], int], float],
) -> None:
    """Перевіряє ручну формулу норми PyTorch."""
    student = load_student_implementation()

    result = student.manual_lp_norm_torch(to_tensor(vector_case), norm_order)

    assert isinstance(result, torch.Tensor)
    assert result.ndim == 0
    assert result.item() == pytest.approx(
        reference_lp_norm(vector_case, norm_order)
    )


def test_row_norms_torch(
    matrix_data: np.ndarray,
    norm_order: int,
    reference_row_norms: Callable[[np.ndarray, int], np.ndarray],
) -> None:
    """Перевіряє норми рядків PyTorch із dim=1."""
    student = load_student_implementation()

    result = student.row_norms_torch(to_tensor(matrix_data), norm_order)

    assert isinstance(result, torch.Tensor)
    assert result.shape == (matrix_data.shape[0],)
    assert_close(result.numpy(), reference_row_norms(matrix_data, norm_order))


def test_compare_libraries(
    centered_iris_features: np.ndarray,
    norm_orders: tuple[int, ...],
    reference_row_norms: Callable[[np.ndarray, int], np.ndarray],
    capsys: pytest.CaptureFixture[str],
) -> None:
    """Перевіряє збіг бібліотек і результат для кожного порядку."""
    student = load_student_implementation()
    title = "Центровані ознаки Iris"

    results = student.compare_libraries(centered_iris_features, title)

    assert tuple(results) == norm_orders
    output = capsys.readouterr().out
    assert title in output
    for order in norm_orders:
        expected = reference_row_norms(centered_iris_features, order)
        assert_close(results[order], expected)
        assert f"Норма L{order}:" in output
        assert f"рядок={np.argmax(expected)}" in output
    for library in LIBRARIES:
        assert output.count(f"{library}: рядок=") == len(norm_orders)
```

```
def test_compare_libraries_detects_mismatch(
    matrix_data: np.ndarray,
    monkeypatch: pytest.MonkeyPatch,
) -> None:
    """Перевіряє, що розбіжність між бібліотеками не залишиться непоміченою."""
    student = load_student_implementation()
    row_norms_scipy = student.row_norms_scipy

    def shifted_norms(data: np.ndarray, order: int) -> np.ndarray:
        return row_norms_scipy(data, order) + 1e-3

    monkeypatch.setattr(student, "row_norms_scipy", shifted_norms)

    with pytest.raises(AssertionError):
        student.compare_libraries(matrix_data, "Розбіжність бібліотек")


def test_top_five_observations(
    iris_dataset: Any,
    centered_iris_features: np.ndarray,
    norm_order: int,
    reference_row_norms: Callable[[np.ndarray, int], np.ndarray],
) -> None:
    """Перевіряє індекси, види та оцінки п'яти найбільших норм."""
    student = load_student_implementation()
    scores = reference_row_norms(centered_iris_features, norm_order)

    observations = student.top_five_observations(
        scores,
        iris_dataset.target,
        iris_dataset.target_names,
    )

    assert len(observations) == 5
    indices = [index for index, _, _ in observations]
    assert len(set(indices)) == 5
    for index, species, score in observations:
        assert species == iris_dataset.target_names[iris_dataset.target[index]]
        assert score == pytest.approx(scores[index])
    # Послідовність оцінок має збігатися з п'ятьма найбільшими за спаданням.
    assert_close(
        [score for _, _, score in observations],
        sorted(scores, reverse=True)[:5],
    )


def test_top_five_observations_reject_invalid_input(
    iris_dataset: Any,
    centered_iris_features: np.ndarray,
    reference_row_norms: Callable[[np.ndarray, int], np.ndarray],
) -> None:
    """Перевіряє відхилення неузгоджених оцінок і міток."""
    student = load_student_implementation()
    scores = reference_row_norms(centered_iris_features, 2)
    labels = iris_dataset.target
    target_names = iris_dataset.target_names

    with pytest.raises(ValueError, match="одновимірними"):
        student.top_five_observations(
            scores.reshape(-1, 1),
            labels,
            target_names,
```

```
        )
    with pytest.raises(ValueError, match="Кількість оцінок"):
        student.top_five_observations(scores[:-1], labels, target_names)


def test_norm_methods_reject_unsupported_order(
    invalid_norm_order: int,
    vector_case: np.ndarray,
    matrix_data: np.ndarray,
) -> None:
    """Перевіряє ValueError для непідтримуваного порядку норми."""
    student = load_student_implementation()
    calls = (
        (student.manual_lp_norm_numpy, vector_case),
        (student.vector_norm_scipy, vector_case),
        (student.manual_lp_norm_torch, to_tensor(vector_case)),
        (student.row_norms_numpy, matrix_data),
        (student.row_norms_scipy, matrix_data),
        (student.row_norms_sklearn, matrix_data),
        (student.row_norms_torch, to_tensor(matrix_data)),
    )

    for method, values in calls:
        with pytest.raises(ValueError, match="Порядок норми"):
            method(values, invalid_norm_order)


def test_vector_norm_methods_reject_matrix(
    matrix_data: np.ndarray,
) -> None:
    """Перевіряє відхилення матриці функціями векторних норм."""
    student = load_student_implementation()
    calls = (
        (student.manual_lp_norm_numpy, matrix_data),
        (student.vector_norm_scipy, matrix_data),
        (student.manual_lp_norm_torch, to_tensor(matrix_data)),
    )

    for method, values in calls:
        with pytest.raises(ValueError, match="одновимірний вектор"):
            method(values, 2)


def test_row_norm_methods_reject_vector(
    vector_case: np.ndarray,
) -> None:
    """Перевіряє відхилення вектора функціями нормування рядків."""
    student = load_student_implementation()
    calls = (
        (student.row_norms_numpy, vector_case),
        (student.row_norms_scipy, vector_case),
        (student.row_norms_sklearn, vector_case),
        (student.row_norms_torch, to_tensor(vector_case)),
    )

    for method, values in calls:
        with pytest.raises(ValueError, match="двовимірна матриця"):
            method(values, 2)
```

Додаток В. Результат запуску pytest

Наведено вивід команди `python -m pytest -v` для файлу `tests/test_student_implementation.py` без спільного префікса шляху. Рядки решти 141 тесту, які також пройшли, опущено.

```
test_manual_lp_norm_numpy[mixed-signs-L1] PASSED
test_manual_lp_norm_numpy[mixed-signs-L2] PASSED
test_manual_lp_norm_numpy[mixed-signs-L3] PASSED
test_manual_lp_norm_numpy[small-vector-L1] PASSED
test_manual_lp_norm_numpy[small-vector-L2] PASSED
test_manual_lp_norm_numpy[small-vector-L3] PASSED
test_manual_lp_norm_numpy[equal-coordinates-L1] PASSED
test_manual_lp_norm_numpy[equal-coordinates-L2] PASSED
test_manual_lp_norm_numpy[equal-coordinates-L3] PASSED
test_row_norms_numpy[L1] PASSED
test_row_norms_numpy[L2] PASSED
test_row_norms_numpy[L3] PASSED
test_row_norms_scipy[L1] PASSED
test_row_norms_scipy[L2] PASSED
test_row_norms_scipy[L3] PASSED
test_vector_norm_scipy[mixed-signs-L1] PASSED
test_vector_norm_scipy[mixed-signs-L2] PASSED
test_vector_norm_scipy[mixed-signs-L3] PASSED
test_vector_norm_scipy[small-vector-L1] PASSED
test_vector_norm_scipy[small-vector-L2] PASSED
test_vector_norm_scipy[small-vector-L3] PASSED
test_vector_norm_scipy[equal-coordinates-L1] PASSED
test_vector_norm_scipy[equal-coordinates-L2] PASSED
test_vector_norm_scipy[equal-coordinates-L3] PASSED
test_scale_features[unit-interval] PASSED
test_scale_features[symmetric-interval] PASSED
test_row_norms_sklearn[L1] PASSED
test_row_norms_sklearn[L2] PASSED
test_row_norms_sklearn[L3] PASSED
test_manual_lp_norm_torch[mixed-signs-L1] PASSED
test_manual_lp_norm_torch[mixed-signs-L2] PASSED
test_manual_lp_norm_torch[mixed-signs-L3] PASSED
test_manual_lp_norm_torch[small-vector-L1] PASSED
test_manual_lp_norm_torch[small-vector-L2] PASSED
test_manual_lp_norm_torch[small-vector-L3] PASSED
test_manual_lp_norm_torch[equal-coordinates-L1] PASSED
test_manual_lp_norm_torch[equal-coordinates-L2] PASSED
test_manual_lp_norm_torch[equal-coordinates-L3] PASSED
test_row_norms_torch[L1] PASSED
test_row_norms_torch[L2] PASSED
test_row_norms_torch[L3] PASSED
test_compare_libraries PASSED
test_compare_libraries_detects_mismatch PASSED
test_top_five_observations[L1] PASSED
test_top_five_observations[L2] PASSED
test_top_five_observations[L3] PASSED
test_top_five_observations_reject_invalid_input PASSED
test_norm_methods_reject_unsupported_order[zero-mixed-signs] PASSED
test_norm_methods_reject_unsupported_order[zero-small-vector] PASSED
test_norm_methods_reject_unsupported_order[zero-equal-coordinates] PASSED
test_norm_methods_reject_unsupported_order[negative-mixed-signs] PASSED
```

```
test_norm_methods_reject_unsupported_order[negative-small-vector] PASSED
test_norm_methods_reject_unsupported_order[negative-equal-coordinates] PASSED
test_norm_methods_reject_unsupported_order[unsupported-mixed-signs] PASSED
test_norm_methods_reject_unsupported_order[unsupported-small-vector] PASSED
test_norm_methods_reject_unsupported_order[unsupported-equal-coordinates] PASSED
test_vector_norm_methods_reject_matrix PASSED
test_row_norm_methods_reject_vector[mixed-signs] PASSED
test_row_norm_methods_reject_vector[small-vector] PASSED
test_row_norm_methods_reject_vector[equal-coordinates] PASSED

201 passed in 2.05s
```